Module 2

Supervised Learning Algorithms and Model Optimization

Regression Algorithms

In machine learning, regression analysis is a statistical technique that predicts continuous numeric values based on the relationship between independent and dependent variables. The main goal of regression analysis is to plot a line or curve that best fit the data and to estimate how one variable affects another.

Regression analysis is a fundamental concept in machine learning and it is used in many applications such as forecasting, predictive analytics, etc.

In machine learning, regression is a type of supervised learning. The key objective of regression-based tasks is to predict output labels or responses, which are continuous numeric values, for the given input data. The output will be based on what the model has learned in the training phase.

Regression models use the input data features (independent variables) and their corresponding continuous numeric output values (dependent or outcome variables) to learn specific associations between inputs and corresponding outputs.
Regression in machine learning is a supervised learning. Basically, regression is a statistical technique that finds a relationship between dependent and independent variables. To implement regression in machine learning, a regression algorithm is trained with a labeled dataset. The dataset contains features (independent variables) and target values (dependent variable).

During the training phase, the regression algorithm learns the relation between independent variables (predictors) and dependent variables (target).

The regression models predict new values based on the learned relation between predictors and targets during the training.

Types of Regression in Machine Learning

Generally, the classification of regression methods is done based on the three metrics − the number of independent variables, type of dependent variables, and shape of the regression line.

There are numerous regression techniques used in machine learning. However, the following are commonly used types of regression −

Linear Regression
Logistic Regression
Polynomial Regression
Lasso Regression
Ridge Regression
Decision Tree Regression
Random Forest Regression
Support Vector Regression

Linear Regression

Linear regression is the most commonly used regression model in machine learning. It may be defined as the statistical model that analyzes the linear relationship between a dependent variable with a given set of independent variables. A linear relationship between variables means that when the value of one or more independent variables changes (increase or decrease), the value of the dependent variable will also change accordingly (increase or decrease).

Linear regression is further divided into two subcategories: simple linear regression and multiple linear regression (also known as multivariate linear regression).

In simple linear regression, a single independent variable (or predictor) is used to predict the dependent variable.

Mathematically, the simple linear regression can be represented as follows −

$Y=mX+b$

Where,

Y is the dependent variable we are trying to predict. X is the dependent variable we are using to make predictions.
m is the slope of the regression line, which represents the effect X has on Y. b is a constant known as the Y-intercept. If X = 0, Y would be equal to b
.

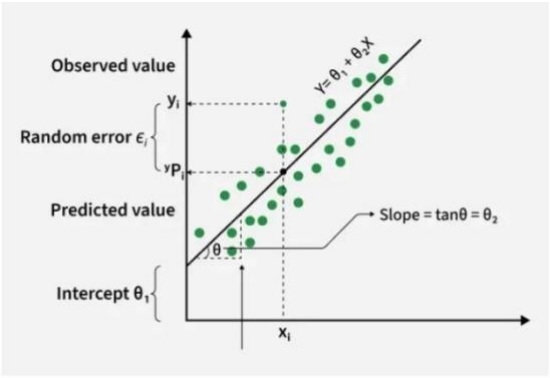
Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. It predicts continuous values by fitting a straight line that best represents the data.
It assumes that there is a linear relationship between the input and output
Uses a best-fit line to make predictions
Commonly used in forecasting, trend analysis, and predictive modelling

Best Fit Line

In linear regression, the best-fit line is the straight line that best represents the relationship between the independent variable (input) and the dependent variable (output). The goal is to minimize the difference between the actual data points and the predicted values generated by the model.

1.Goal of the Best-Fit Line

The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values. This line helps us predict the dependent variable for new, unseen data.



Linear Regression

Here Y is called a dependent or target variable and X is called an independent variable also known as the predictor of Y.

- θ1 represents the intercept, which is the value of Y when X = 0
- θ2 represents the slope, which shows how much Y changes for a unit change in X

There are many types of functions or modules that can be used for regression. A linear function is the simplest type of function. Here, X may be a single feature or multiple features representing the problem.

2. Equation of the Best-Fit Line

For simple linear regression (with one independent variable), the best-fit line is represented by the equation

$y=mx+b$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

3. Minimizing the Error using the Least Squares Method

To determine the best-fit line, linear regression uses the Least Squares Method, which minimizes the difference between actual and predicted values. These differences are called residuals. These differences are called residuals.

The formula for residuals is:

$$Residual = y_i - \hat{y}_i$$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$\Sigma(y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

4. Interpretation of the Best-Fit Line

- **Slope (m):** The slope indicates how much the dependent variable changes for every one-unit increase in the independent variable. For example, if the slope is 5, then y increases by 5 units for every 1-unit increase in x.
- **Intercept (b):** The intercept represents the predicted value of y when x = 0. It's the point where the line crosses the y-axis.

In linear regression some hypothesis are made to ensure reliability of the model's results.

Limitations:

- ***Assumes Linearity:*** *The method assumes the relationship between the variables is linear. If the relationship is non-linear, linear regression might not work well.*
- ***Sensitivity to Outliers:*** *Outliers can significantly affect the slope and intercept, skewing the best-fit line.*

Hypothesis Function

In linear regression, the hypothesis function is the equation used to make predictions about the dependent variable based on the independent variables. It represents the relationship between the input features and the target output.

For a simple case with one independent variable, the hypothesis function is:

$h(x)=\beta_0+\beta_1 x$

Where:

- $h(x)$ or $(y^)$ is the predicted value of the dependent variable (y).
- X is the independent variable.
- β_0 is the intercept, representing the value of y when x is 0.
- β_1 is the slope, indicating how much y changes for each unit change in x.

For multiple linear regression (with more than one independent variable), the hypothesis function expands to:

$$h(x_1,x_2,...,x_k)=\beta_0+\beta_1x_1+\beta_2x_2+...+\beta_k x_k$$

Where:

- $x_1,x_2,...,x_k$ are the independent variables.
- β_0 is the intercept.
- $\beta_1,\beta_2,...,\beta_k$ are the coefficients, representing the influence of each respective independent variable on the predicted output.

Cost Function

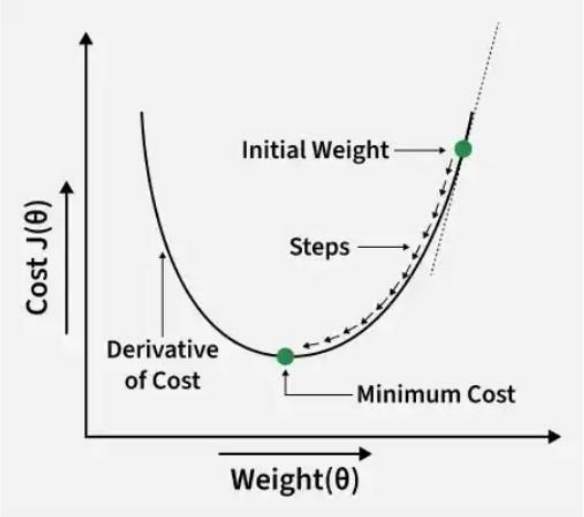
In Linear Regression, the cost function measures how far the predicted values $Y^Y^$ are from the actual values (Y). It helps identify and reduce errors to find the best-fit line. The most common cost function used is Mean Squared Error (MSE), which calculates the average of squared differences between actual and predicted values.

$$\text{Cost function}(J) = \frac{1}{n}\sum_n^i(\hat{y}_i - y_i)^2$$

$\hat{y}_i = \theta_1 + \theta_2 x_i$ It is used to minimize this cost, we use Gradient Descent, which iteratively updates θ_1 and θ_2 until the MSE reaches its lowest value. This ensures the line fits the data as accurately as possible.

Gradient Descent

Gradient descent is an optimization technique used to train a linear regression model by minimizing the prediction error. It works by starting with random model parameters and repeatedly adjusting them to reduce the difference between predicted and actual values.



Gradient Descent

How it works:

- Start with random values for slope and intercept.
- Calculate the error between predicted and actual values.
- Find how much each parameter contributes to the error (gradient).
- Update the parameters in the direction that reduces the error.
- Repeat until the error is as small as possible.

This helps the model find the best-fit line for the data.

Types of Linear Regression

When there is only one independent feature it is known as Simple Linear Regression or Univariate Linear Regression and when there are more than one feature it is known as Multiple Linear Regression or Multivariate Regression.

1. Simple Linear Regression

Simple linear regression is used when we want to predict a target value (dependent variable) using only one input feature (independent variable). It assumes a straight-line relationship between the two.

$$\hat{y} = \theta_0 + \theta_1 x$$

Where:

- y^$\hat{y}$ is the predicted value
- xx is the input (independent variable)
- θ0θ_0 is the intercept (value of y^$\hat{y}$ when x=0)
- θ1θ_1 is the slope or coefficient (how much y^$\hat{y}$ changes with one unit of x) **Example:** Predicting a person's salary (y) based on their years of experience (x).

2. Multiple Linear Regression

Multiple linear regression involves more than one independent variable and one dependent variable. The equation for multiple linear regression is:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Where:

- y^ is the predicted value
- x1,x2,…,xn are the independent variables
- θ1,θ2,…,θn are the coefficients (weights) corresponding to each predictor
- θ_0 is the intercept

The goal is to find the best-fit line that predicts Y accurately for given inputs X.

Use Cases:

- **Real Estate:** Predict property prices using location, size and other factors.
- **Finance:** Forecast stock prices using interest rates and inflation data.
- **Agriculture:** Estimate crop yield from rainfall, temperature and soil quality.
- **E-commerce:** Analyze how price, promotions and seasons affect sales.

Polynomial Regression

Polynomial Regression is a form of linear regression where the relationship between the independent variable (x) and the dependent variable (y) is modelled as an n_{th} degree polynomial. It is useful when the data exhibits a non-linear relationship allowing the model to fit a curve to the data.

Need for Polynomial Regression

- **Non-linear Relationships:** Polynomial regression is used when the relationship between the independent variable (input) and dependent variable (output) is non-linear. Unlike linear regression which fits a straight line, it fits a polynomial equation to capture the curve in the data.
- **Better Fit for Curved Data:** When a researcher hypothesizes a curvilinear relationship, polynomial terms are added to the model. A linear model often results in residuals with noticeable patterns which shows a poor fit. It can capture these non-linear patterns effectively.
- **Flexibility and Complexity:** It does not assume all independent variables are independent. By introducing higher-degree terms, it allows for more flexibility and can model more complex, curvilinear relationships between variables.

How does a Polynomial Regression work

Polynomial regression is an extension of linear regression where higher-degree terms are added to model non-linear relationships. The general form of the equation for a polynomial regression of degree n is:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_n x^n + \epsilon$$

where:

- y is the dependent variable.
- x is the independent variable.
- $\beta_0,\beta_1,\ldots,\beta_n$ are the coefficients of the polynomial terms.
- n is the degree of the polynomial.

- ϵ represents the error term.

The goal of regression analysis is to model the expected value of a dependent variable y in terms of an independent variable x. In simple linear regression, this relationship is modeled as: $y=a+bx+e$

Here

- y is a dependent variable
- a is the y-intercept, b is the slope
- e is the error term

However in cases where the relationship between the variables is nonlinear such as modelling chemical synthesis based on temperature, a linear model may not be sufficient. Instead, we use polynomial regression which introduces higher-degree terms such as x_2 to better capture the relationship.

For example, a quadratic model can be written as:

$$y = a + b_1x + b_2x^2 + e$$

Here:

- y is the dependent variable on x
- $a$is the y-intercept and e is the error rate.

In general, polynomial regression can be extended to the nth degree:

$$y = a + b_1x + b_2x^2 + ... + b_nx^n$$

While the regression function is linear in terms of the unknown coefficients $b_0,b_1,\ldots,b_n$, the model itself captures non-linear patterns in the data. The coefficients are estimated using techniques like Least Square technique to minimize the error between predicted and actual values.

Choosing the right polynomial degree n is important: a higher degree may fit the data more closely but it can lead to overfitting. The degree should be selected based on the complexity of the data. Once the model is trained, it can be used to make predictions on new data, capturing non-linear relationships and providing a more accurate model for real-world applications.

Real-Life Example for Polynomial Regression

Let’s consider an example in the field of finance where we analyze the relationship between an employee's years of experience and their corresponding salary. If we check that the relationship might not be linear, polynomial regression can be used to model it more accurately.

Example Data:

Years of Experience	Salary (in dollars)
1	50,000

2	55,000

Years of Experience	**Salary (in dollars)**
3	65,000
4	80,000
5	110,000
6	150,000
7	200,000

Now, let's apply polynomial regression to model the relationship between years of experience and salary. We'll use a quadratic polynomial (degree 2) which includes both linear and quadratic terms for better fit. The quadratic polynomial regression equation is:

$$Salary = \beta_0 + \beta_1 \times Experience + \beta_2 \times Experience^2 + \epsilon$$

To find the coefficients β_0,β_1,β_2 that minimize the difference between the predicted and actual salaries, we can use the Least Squares method**.** The objective is to minimize the sum of squared

differences between the predicted salaries and the actual data points which allows us to fit a model that captures the non-linear progression of salary with respect to experience.

Evaluation Metrics: MSE, RMSE, R^2

Regression is a supervised learning technique used to model and analyze the relationship between input variables (features) and a continuous output variable (target). The primary objective of a regression model is to make accurate numerical predictions.

- They quantify the prediction error of regression models
- Different metrics emphasize different error characteristics
- Some metrics penalize large errors more than small ones
- Metric selection depends on problem requirements and data nature
- They help in model selection and optimization

Types of Regression Metrics

1. Mean Absolute Error (MAE)

2. Mean Squared Error (MSE)

Mean Squared Error calculates the average of squared differences between actual and predicted values. By squaring errors, it penalizes larger mistakes more strongly, making it sensitive to outliers.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- n = number of observations
- y_i = actual value
- $\hat{y}_i$ = predicted value

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

Output:

```
MSE: 2900.193628493482
```

3. Root Mean Squared Error (RMSE)

Root Mean Squared Error is the square root of MSE. It maintains the strong penalty for large errors while converting the result back to the original unit of the target variable, improving interpretability.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Where:

- n = number of observations
- y_i = actual value

- $\hat{y}_i$ = predicted value

```
import numpy as np
from sklearn.metrics import mean_squared_error

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("RMSE:", rmse)
```

Output:

```
RMSE: 53.85344583676593
```

4. R-squared (R^2 Score)

R-squared represents the proportion of variance in the target variable that is explained by the regression model. It provides insight into how well the model captures underlying data patterns.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where:

- $\bar{y}$ = mean of actual values
- Numerator = residual sum of squares
- Denominator = total sum of squares

```
from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)
print("R² Score:", r2)
```

Output:

```
R² Score: 0.4526027629719195
```

5. Mean Absolute Percentage Error (MAPE)

Classification in Machine Learning

Classification may be defined as the process of predicting class or category from observed values or given data points. The categorized output can have the form such as "Black" or "White" or "spam" or "no spam".

Classification in machine learning is a supervised learning technique where an algorithm is trained with labeled data to predict the category of new data.

Mathematically, classification is the task of approximating a mapping function (f) from input variables (X) to output variables (Y). It is basically belongs to the supervised machine learning in which targets are also provided along with the input data set.

An example of classification problem can be the spam detection in emails. There can be only two categories of output, "spam" and "no spam"; hence this is a binary type classification.

To implement this classification, we first need to train the classifier. For this example, "spam" and "no spam" emails would be used as the training data. After successfully train the classifier, it can be used to detect an unknown email.
Types of Learners in Classification
We have two types of learners in respective to classification problems −

Lazy Learners − As the name suggests, such kind of learners waits for the testing data to be appeared after storing the training data. Classification is done only after getting the testing data. They spend less time on training but more time on predicting. Examples of lazy learners are K-nearest neighbor and case-based reasoning.
Eager Learners − As opposite to lazy learners, eager learners construct classification model without waiting for the testing data to be appeared after storing the training data. They spend more time on training but less time on predicting. Examples of eager learners are Decision Trees, Nave Bayes and Artificial Neural Networks (ANN).

Classification Algorithms in Machine Learning
The classification algorithm is a type of supervised learning technique that involves predicting a categorical target variable based on a set of input features. It is commonly used to solve problems such as spam detection, fraud detection, image recognition, sentiment analysis, and many others.

The goal of a classification model is to learn a mapping function (f) between the input features (X) and the target variable (Y). This mapping function is often represented as a decision boundary, which separates different classes in the input feature space. Once the model is trained, it can be used to predict the class of new, unseen examples.

To implement a classification model, it is important to understand the algorithms used for classification. One of the most commonly used algorithms is Logistic Regression. Classification algorithms can be grouped into different categories, such as:

1. Linear Classifiers
Linear classifier models create a linear decision boundary between classes. They are simple and computationally efficient. Some of the linear classification models are as follows:

Logistic Regression
Support Vector Machines
Single-layer Perceptron
Stochastic Gradient Descent (SGD) Classifier

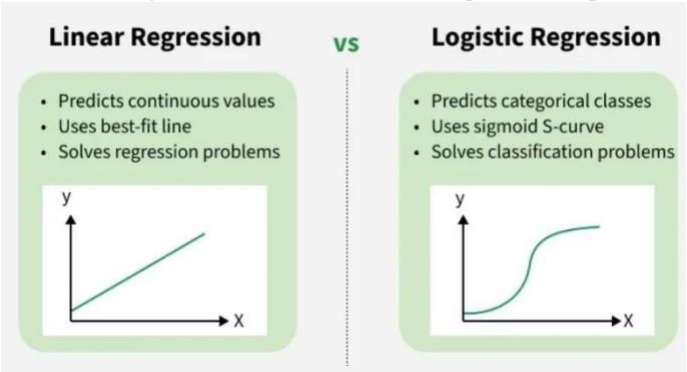
2. Non linear Classifiers
Non linear models create non linear decision boundaries to separate classes. They can capture more complex relationships between input features and the target variable. Some common non linear classification models include:

K-Nearest Neighbours
Kernel SVM
Decision Tree Classification
Random Forests
AdaBoost
Bagging Classifier
Voting Classifier
Extra Trees Classifier

Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression, which predicts continuous values it predicts the probability that an input belongs to a specific class.

- It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.
- It uses sigmoid function to convert inputs into a probability value between 0 and 1.



Types

1. **Binomial Logistic Regression:** Used when the dependent variable has only two possible categories, such as Yes/No, Pass/Fail, or 0/1. It is the most common type and is used for binary classification tasks.
2. **Multinomial Logistic Regression:** Used when the dependent variable has three or more unordered categories. For example, classifying animals as cat, dog, or sheep. It extends logistic regression to handle multiple classes.
3. **Ordinal Logistic Regression:** Used when the dependent variable has three or more categories with a natural order, such as Low, Medium, and High. It considers the ranking of categories during prediction.

Assumptions

1. **Independent Observations:** Each data point should be independent of the others, meaning there should be no dependence between observations.
2. **Binary Dependent Variable:** The target variable is typically binary and can take only two values, such as 0/1 or Yes/No. For multiclass problems, extensions such as Softmax-based logistic regression are used.
3. **Linearity of Log-Odds:** The independent variables should have a linear relationship with the log-odds of the dependent variable.
4. **No Extreme Outliers:** Extreme outliers can distort coefficient estimates and negatively affect model performance.
5. **Large Sample Size:** A sufficiently large dataset helps produce more reliable and stable predictions.

Understanding Sigmoid Function

1. The sigmoid function is a key component of Logistic Regression that converts the model's raw output into a probability value between 0 and 1.

2. This function takes any real number and maps it into the range 0 to 1 forming an "S" shaped curve called the sigmoid curve or logistic curve. Because probabilities must lie between 0 and 1, the sigmoid function is perfect for this purpose.

3. In logistic regression, we use a threshold value usually 0.5 to decide the class label.

- If the sigmoid output is same or above the threshold, the input is classified as Class 1.
- If it is below the threshold, the input is classified as Class 0.

This approach helps to transform continuous input values into meaningful class predictions.

Working

Logistic regression computes a linear combination of input features ($z = w \cdot X + b$) and passes it through a sigmoid function to produce a probability between 0 and 1. This probability is then used to assign the input to a class.

Suppose we have input features represented as a matrix:

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1m} \\ x_{21} & \cdots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nm} \end{bmatrix}$$

and the dependent variable is Y having only binary value i.e 0 or 1.

$$Y = \begin{cases} 0 & \text{if } Class\ 1 \\ 1 & \text{if } Class\ 2 \end{cases}$$

then, apply the multi-linear function to the input variables X.

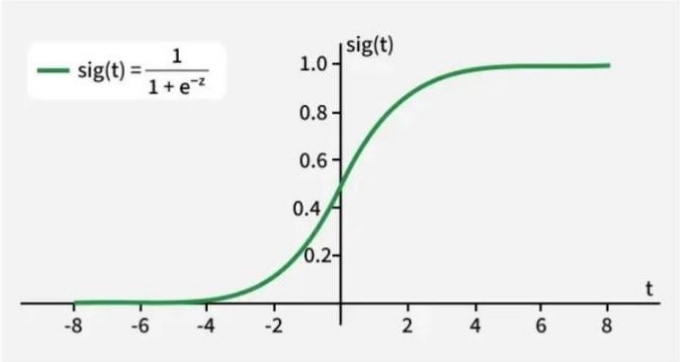
$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b$$

Here x_i is the ith observation of X, $w_i = [w_1, w_2, w_3, \cdots, w_m]$ is the weights or Coefficient and b is the bias term also known as intercept. Simply this can be represented as the dot product of weight and bias. $z = w \cdot X + b$

At this stage, z is a continuous value from the linear regression. Logistic regression then applies the sigmoid function to z to convert it into a probability between 0 and 1 which can be used to predict the class.

Now we use the sigmoid function where the input will be z and we find the probability between 0 and 1. i.e. predicted y.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Sigmoid function

As shown above the sigmoid function converts the continuous variable data into the probability i.e between 0 and 1.

- $\sigma(z)$ tends towards 1 as $z \to \infty$
- $\sigma(z)$ tends towards 0 as $z \to -\infty$
- $\sigma(z)$ is always bounded between 0 and 1

where the probability of being a class can be measured as:

$$P(y = 1) = \sigma(z)$$
$$P(y = 0) = 1 - \sigma(z)$$

Implementation

1. Binomial Logistic regression:

In binomial logistic regression, the target variable can only have two possible values such as "0" or "1", "pass" or "fail". The sigmoid function is used for prediction.

We will be using Scikit-learn library for this and shows how to use the breast cancer dataset to implement a Logistic Regression model for classification.

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=23)

clf = LogisticRegression(max_iter=10000, random_state=0)
clf.fit(X_train, y_train)

acc = accuracy_score(y_test, clf.predict(X_test)) * 100
print(f"Logistic Regression model accuracy: {acc:.2f}%")
```

Output:

```
Logistic Regression model accuracy (in %): 96.49%
```

2. Multinomial Logistic Regression:

Target variable can have 3 or more possible types which are not ordered i.e types have no quantitative significance like “disease A” vs “disease B” vs “disease C”.

In this case, the softmax function is used in place of the sigmoid function. Softmax function for K classes will be:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Here K represents the number of elements in the vector z and i, j iterates over all the elements in the vector.

Then the probability for class c will be:

$$P(Y = c|\overrightarrow{X} = x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{k=1}^{K} e^{w_k \cdot x + b_k}}$$

```python
from sklearn.model_selection import train_test_split
from sklearn import datasets, linear_model, metrics

digits = datasets.load_digits()

X = digits.data
y = digits.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=1)

reg = linear_model.LogisticRegression(max_iter=10000, random_state=0)
reg.fit(X_train, y_train)

y_pred = reg.predict(X_test)

print(f"Logistic Regression model accuracy: {metrics.accuracy_score(y_test, y_pred) * 100:.2f}%")
```

Output:

Logistic Regression model accuracy: 96.66%

Differences Between Linear and Logistic Regression

Aspect	Linear Regression	Logistic Regression
Definition	Linear regression is used to predict the continuous dependent variable using a given set of independent variables.	Logistic regression is used to predict the categorical dependent variable using a given set of independent variables.
Problem Type	It is used for solving regression problem.	It is used for solving classification problems.
Output Type	In this we predict the value of continuous variables.	In this we predict values of categorical variables.
Curve/Model Fitting	In this we find best fit line.	In this we find S-Curve.
Estimation Method	Least square estimation method is used for estimation of accuracy.	Maximum likelihood estimation method is used for estimation of accuracy.
Output Example	The output must be continuous value such as price, age etc.	Output must be categorical value such as 0 or 1, Yes or No, etc.

K-Nearest Neighbors (KNN)

K-nearest neighbors (KNN) algorithm is a type of supervised ML algorithm which can be used for both classification as well as regression predictive problems. However, it is mainly used for classification predictive problems in industry. The main idea behind KNN is to find the k-nearest data points to a given test data point and use these nearest neighbors to make a prediction. The value of k is a hyperparameter that needs to be tuned, and it represents the number of neighbors to consider.

For classification problems, the KNN algorithm assigns the test data point to the class that appears most frequently among the k-nearest neighbors. In other words, the class with the highest number of neighbors is the predicted class.

For regression problems, the KNN algorithm assigns the test data point the average of the k-nearest neighbors' values.

The distance metric used to measure the similarity between two data points is an essential factor that affects the KNN algorithm's performance. The most commonly used distance metrics are Euclidean distance, Manhattan distance, and Minkowski distance.

The following two properties would define KNN well −

Lazy learning algorithm − KNN is a lazy learning algorithm because it does not have a specialized training phase and uses all the data for training while classification.

Non-parametric learning algorithm − KNN is also a non-parametric learning algorithm because it doesn't assume anything about the underlying data.
Working of KNN

K-nearest neighbors (KNN) algorithm uses 'feature similarity' to predict the values of new datapoints which further means that the new data point will be assigned a value based on how closely it matches the points in the training set. We can understand its working with the help of following steps −

Step 1 − For implementing any algorithm, we need dataset. So during the first step of KNN, we must load the training as well as test data.

Step 2 − Next, we need to choose the value of K i.e. the nearest data points. K can be any integer.

Step 3 − For each point in the test data do the following −

3.1 − Calculate the distance between test data and each row of training data with the help of any of the method namely: Euclidean, Manhattan or Hamming distance. The most commonly used method to calculate distance is Euclidean.

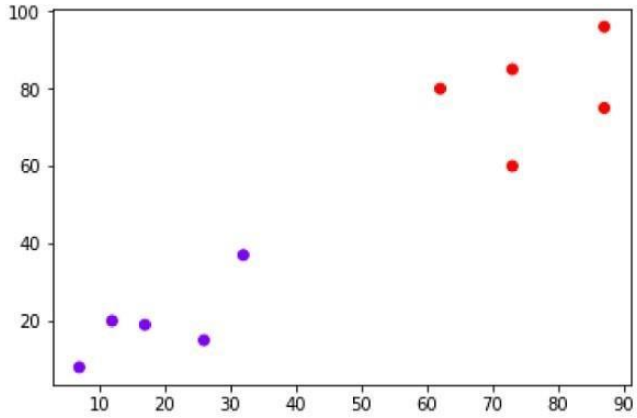
3.2 − Now, based on the distance value, sort them in ascending order.

3.3 − Next, it will choose the top K rows from the sorted array.

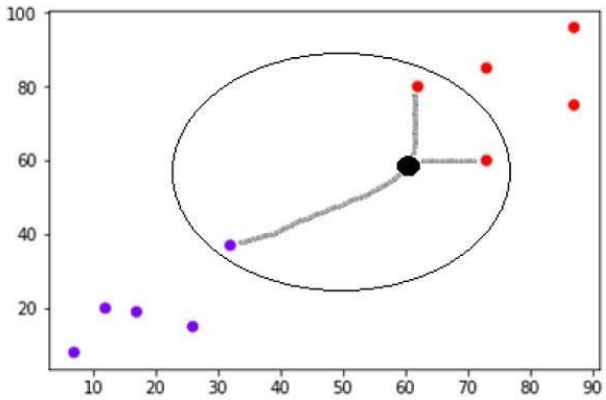
3.4 − Now, it will assign a class to the test point based on most frequent class of these rows.

Step 4 − End
Example
The following is an example to understand the concept of K and working of KNN algorithm −

Suppose we have a dataset which can be plotted as follows −



Now, we need to classify new data point with black dot (at point 60,60) into blue or red class. We are assuming K = 3 i.e. it would find three nearest data points. It is shown in the next diagram −



We can see in the above diagram the three nearest neighbors of the data point with black dot. Among those three, two of them lies in Red class hence the black dot will also be assigned in red class.
Building a K Nearest Neighbors Model
We can follow the below steps to build a KNN model −

Load the data − The first step is to load the dataset into memory. This can be done using various libraries such as pandas or numpy.

Split the data − The next step is to split the data into training and test sets. The training set is used to train the KNN algorithm, while the test set is used to evaluate its performance.

Normalize the data − Before training the KNN algorithm, it is essential to normalize the data to ensure that each feature contributes equally to the distance metric calculation.

Calculate distances − Once the data is normalized, the KNN algorithm calculates the distances between the test data point and each data point in the training set.

Select k-nearest neighbors − The KNN algorithm selects the k-nearest neighbors based on the distances calculated in the previous step.

Make a prediction − For classification problems, the KNN algorithm assigns the test data point to the class that appears most frequently among the knearest neighbors. For regression problems, the KNN algorithm assigns the test data point the average of the k-nearest neighbors' values.

Evaluate performance − Finally, the KNN algorithm's performance is evaluated using various metrics such as accuracy, precision, recall, and F1score.
Data set :- has 8 features columns like i.e *"Age", "Glucose"* e.t.c, and the target variable “Outcome” for 108 patients. So in this, we will create a link Neighbors Classifier model to predict the presence of diabetes or not for patients with such information.

```
# Importing libraries

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from scipy.stats import mode
```

```
from sklearn.neighbors import KNeighborsClassifier
# K Nearest Neighbors Classification
class K_Nearest_Neighbors_Classifier() :
    def __init__( self, K ) :
        self.K = K
    # Function to store training set
    def fit( self, X_train, Y_train ) :
        self.X_train = X_train
        self.Y_train = Y_train
        # no_of_training_examples, no_of_features
        self.m, self.n = X_train.shape
    # Function for prediction
    def predict( self, X_test ) :
        self.X_test = X_test
        # no_of_test_examples, no_of_features
        self.m_test, self.n = X_test.shape
        # initialize Y_predict
        Y_predict = np.zeros( self.m_test )
        for i in range( self.m_test ) :
            x = self.X_test[i]
            # find the K nearest neighbors from current test example
            neighbors = np.zeros( self.K )
            neighbors = self.find_neighbors( x )
```

```
            # most frequent class in K neighbors

            Y_predict[i] = mode( neighbors )[0][0]

        return Y_predict

    # Function to find the K nearest neighbors to current test example

    def find_neighbors( self, x ) :

        # calculate all the euclidean distances between current
        # test example x and training set X_train

        euclidean_distances = np.zeros( self.m )

        for i in range( self.m ) :

            d = self.euclidean( x, self.X_train[i] )

            euclidean_distances[i] = d

        # sort Y_train according to euclidean_distance_array and
        # store into Y_train_sorted

        inds = euclidean_distances.argsort()

        Y_train_sorted = self.Y_train[inds]

        return Y_train_sorted[:self.K]

    # Function to calculate euclidean distance

    def euclidean( self, x, x_train ) :

        return np.sqrt( np.sum( np.square( x - x_train ) ) )

# Driver code

def main() :

    # Importing dataset

    df = pd.read_csv( "diabetes.csv" )
```

```
X = df.iloc[:,:-1].values

Y = df.iloc[:,-1:].values

    # Splitting dataset into train and test set

    X_train, X_test, Y_train, Y_test = train_test_split(
      X, Y, test_size = 1/3, random_state = 0 )

    # Model training

    model = K_Nearest_Neighbors_Classifier( K = 3 )

    model.fit( X_train, Y_train )

    model1 = KNeighborsClassifier( n_neighbors = 3 )

    model1.fit( X_train, Y_train )

    # Prediction on test set

    Y_pred = model.predict( X_test )

    Y_pred1 = model1.predict( X_test )

    # measure performance

    correctly_classified = 0

    correctly_classified1 = 0

    # counter

    count = 0

    for count in range( np.size( Y_pred ) ) :

        if Y_test[count] == Y_pred[count] :

            correctly_classified = correctly_classified + 1

        if Y_test[count] == Y_pred1[count] :

            correctly_classified1 = correctly_classified1 + 1
```

```
        count = count + 1

    print( "Accuracy on test set by our model       :  ", (
correctly_classified / count ) * 100 )     print(
"Accuracy on test set by sklearn model   :  ", (
correctly_classified1 / count ) * 100 )


if __name__ == "__main__" :

    main()
```

Output :

```
Accuracy on test set by our model       :   63.888888888888886
Accuracy on test set by sklearn model   :   63.888888888888886
```

Naive Bayes Classifiers

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorisation and sentiment analysis.

- Original data has two classes: green circles (y = 1) and red squares (y = 2).
- Estimate probability distribution along the first dimension i.e $P(x_1|y=1), P(x_1|y=2)$
- Estimate probability distribution along the second dimension i.e $P(x_2|y=1), P(x_2|y=2)$
- Combine both dimensions using conditional independence i.e $P(x|y) = \prod_{\alpha} P(x_\alpha|y)$

Assumption of Naive Bayes

The fundamental Naive Bayes assumption is that each feature makes an:

- **Feature independence:** This means that when we are trying to classify something, we assume that each feature (or piece of information) in the data does not affect any other feature.
- **Continuous features are normally distributed:** If a feature is continuous, then it is assumed to be normally distributed within each class.
- **Discrete features have multinomial distributions:** If a feature is discrete, then it is assumed to have a multinomial distribution within each class.
- **Features are equally important:** All features are assumed to contribute equally to the prediction of the class label.
- **No missing data:** The data should not contain any missing values.

Types of Naive Bayes Model

There are three types of Naive Bayes Model :

1. Gaussian Naive Bayes

In Gaussian Naive Bayes, continuous values associated with each feature are assumed to be distributed according to a Gaussian distribution. A Gaussian distribution is also called Normal distribution When plotted, it gives a bell shaped curve which is symmetric about the mean of the feature values as shown below:

2. Multinomial Naive Bayes

Multinomial Naive Bayes is used when features represent the frequency of terms (such as word counts) in a document. It is commonly applied in text classification, where term frequencies are important.

3. Bernoulli Naive Bayes

Bernoulli Naive Bayes deals with binary features, where each feature indicates whether a word appears or not in a document. It is suited for scenarios where the presence or absence of terms is more relevant than their frequency. Both models are widely used in document classification tasks

Advantages

- Easy to implement and computationally efficient.
- Effective in cases with a large number of features.
- Performs well even with limited training data.
- It performs well in the presence of categorical features.
- For numerical features data is assumed to come from normal distributions

Disadvantages

- Assumes that features are independent, which may not always hold in real-world data.
- Can be influenced by irrelevant attributes.
- May assign zero probability to unseen events, leading to poor generalization.

Applications

- **Spam Email Filtering**: Classifies emails as spam or non-spam based on features.
- **Text Classification**: Used in sentiment analysis, document categorization and topic classification.
- **Medical Diagnosis:** Helps in predicting the likelihood of a disease based on symptoms.
- **Credit Scoring:** Evaluates creditworthiness of individuals for loan approval.
- **Weather Prediction**: Classifies weather conditions based on various factors.

Decision Boundaries

Classification algorithms separate different groups or classes within data. When visualizing data points on a scatter plot, where each point belongs to a specific category (such as 'spam' or 'not spam', 'cat' or 'dog'), a model must determine which category a new point belongs to. This determination is made using a **decision boundary**.

Think of a decision boundary as an invisible line or surface that the algorithm learns. This boundary divides the space where your data lives into regions, with each region corresponding to a predicted class. If a data point falls on one side of the boundary, the model assigns it to one class; if it falls on the other side, it gets assigned to the other class.

Decision Boundaries in Logistic Regression

In the previous section, we saw that Logistic Regression calculates the probability that a data point belongs to a particular class (let's call it class 1) using the sigmoid function. The output is a probability p between 0 and 1. Typically, we set a threshold, often 0.5, to make the final classification decision:

- If $p \geq 0.5$, predict class 1. •

  If $p < 0.5$, predict class 0.

The decision boundary is precisely where the model is uncertain, meaning the probability is exactly 0.5. When does the sigmoid function output 0.5? It happens when its input is exactly 0.

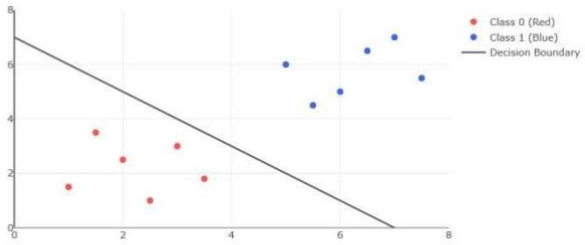
Remember, the input to the sigmoid function in logistic regression is typically a linear combination of the features, like $z = w_0 + w_1 x_1 + w_2 x_2$ for two features (x_1, x_2). So, the decision boundary is defined by the equation:

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

For data with two features, this equation represents a straight line. This line separates the 2D plane into two regions: one where the model predicts class 1 ($z > 0$, so $p > 0.5$) and one where it predicts class 0 ($z < 0$, so $p < 0.5$).

Visualizing a Linear Decision Boundary

Let's make this more concrete. Imagine we have data points belonging to two classes (Red and Blue) plotted based on two features (Feature 1 and Feature 2). A logistic regression model trained on this data might find a linear decision boundary like the one shown below.



A simple scatter plot showing two classes of data points (Red and Blue) separated by a linear decision boundary (the gray line) learned by a model like Logistic Regression. Points generally above and to the right of the line would be classified as Blue (Class 1), while points below and to the left would be classified as Red (Class 0).

Any new data point plotted on this graph would be classified based on which side of the gray line it falls. This visual representation helps understand how the model makes its decisions based on the input features.

Linear Boundaries

It's important to note that decision boundaries aren't always straight lines. While basic Logistic Regression typically produces linear boundaries, classification problems can often require more complex shapes to separate the classes effectively.

Imagine classes that are mixed in a more complicated way, perhaps with one class clustered in the middle and the other forming a ring around it. A straight line wouldn't be very good at separating these. Other algorithms, including the K-Nearest Neighbors (KNN) algorithm we'll discuss next, or modifications to logistic regression (like using polynomial features), can create non-linear decision boundaries (curves, circles, or even more irregular shapes).

Why Decision Boundaries Matter

Understanding decision boundaries helps you:

1. **Visualize Model Behavior:** It gives you a graphical intuition for how the model separates data.

2. **Understand Model Complexity:** A simple boundary (like a straight line) implies a simpler model, while a very wiggly boundary suggests a more complex model.

3. **Identify Potential Issues:** An overly complex boundary that perfectly separates all training points might indicate overfitting (the model learned the training data too well, including noise, and might not perform well on new data). A boundary that fails to separate the classes well might indicate underfitting (the model is too simple).

As we look at different classification algorithms, pay attention to the kinds of decision boundaries they tend to create. This will give you insight into their strengths and weaknesses for different types of data distributions. Next, we'll examine the K-Nearest Neighbors algorithm, which takes a very different approach to classification and results in a different kind of decision boundary.

Machine Learning - Overfitting

Overfitting occurs when a model learns the noise in the training data, rather than the underlying patterns. This causes the model to perform well on the training data, but poorly on new data. Essentially, the model becomes too specialized to the training data, and is unable to generalize to new data.

Overfitting is a common problem when using complex models, such as deep neural networks. These models have many parameters, and are able to fit the training data very closely. However, this often comes at the expense of generalization performance.

Causes of Overfitting

There are several factors that can contribute to overfitting −

Complex models − As mentioned earlier, complex models are more likely to overfit than simpler models. This is because they have more parameters, and are able to fit the training data more closely.

Limited training data − When there is not enough training data, it becomes difficult for the model to learn the underlying patterns, and it may instead learn the noise in the data.

Unrepresentative training data − If the training data is not representative of the problem that the model is trying to solve, the model may learn irrelevant patterns that do not generalize well to new data.

Lack of regularization − Regularization is a technique used to prevent overfitting by adding a penalty term to the cost function. If this penalty term is not present, the model is more likely to overfit.

Techniques to Prevent Overfitting

There are several techniques that can be used to prevent overfitting in machine learning − Cross-validation − Cross-validation is a technique used to evaluate a model's performance on new, unseen data. It involves dividing the data into several subsets, and using each subset in turn as a validation set, while training on the remaining data. This helps to ensure that the model generalizes well to new data.

Early stopping − Early stopping is a technique used to prevent a model from overfitting by stopping the training process before it has converged completely. This is done by monitoring the validation error during training, and stopping when the error stops improving.

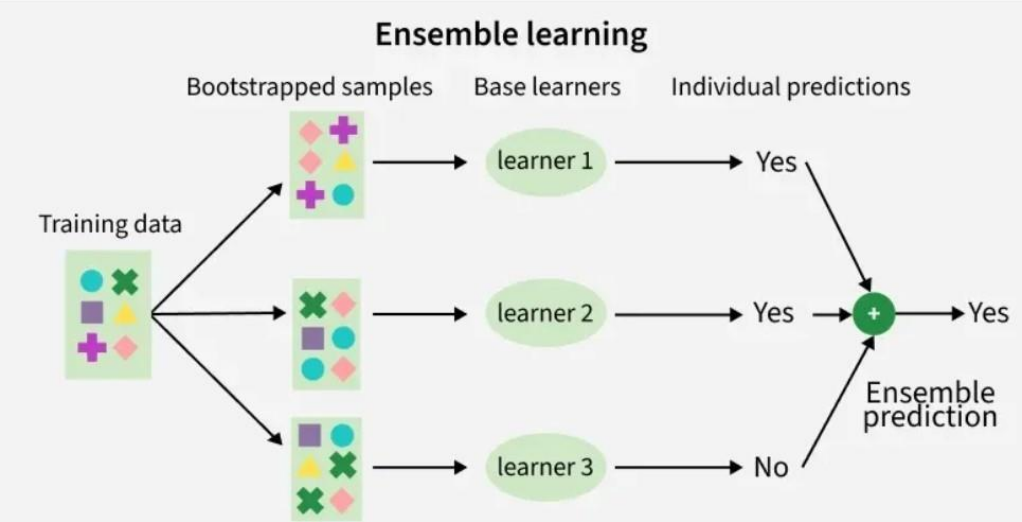
Regularization − Regularization is a technique used to prevent overfitting by adding a penalty term to the cost function. The penalty term encourages the model to have smaller weights, and helps to prevent it from fitting the noise in the training data.

Dropout − Dropout is a technique used in deep neural networks to prevent overfitting. It involves randomly dropping out some of the neurons during training, which forces the remaining neurons to learn more robust features.

Ensemble Learning

Ensemble learning is a method where multiple models are combined instead of using just one. Even if individual models are weak, combining their results gives more accurate and reliable predictions.

- Uses multiple models together to improve overall accuracy.
- Reduces errors by balancing mistakes across models.
- Works on a simple idea similar to combining opinions from a group.



Types of Ensemble Learning

There are three main types of ensemble methods:

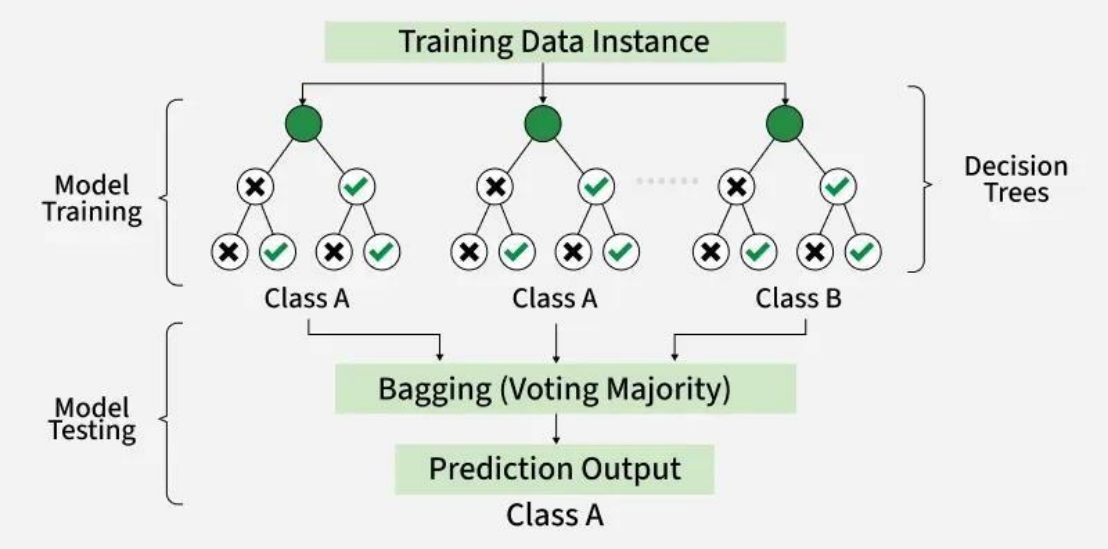
1. **Bagging (Bootstrap Aggregating):** Models are trained independently on different random subsets of the training data. Their results are then combined—usually by averaging (for regression) or voting (for classification). This helps reduce variance and prevents overfitting.
2. **Boosting:** Models are trained one after another. Each new model focuses on fixing the errors made by the previous ones. The final prediction is a weighted combination of all models, which helps reduce bias and improve accuracy.
3. **Stacking (Stacked Generalization):** Multiple different models (often of different types) are trained and their predictions are used as inputs to a final model, called a meta-model. The meta-model learns how to best combine the predictions of the base models, aiming for better performance than any individual model.

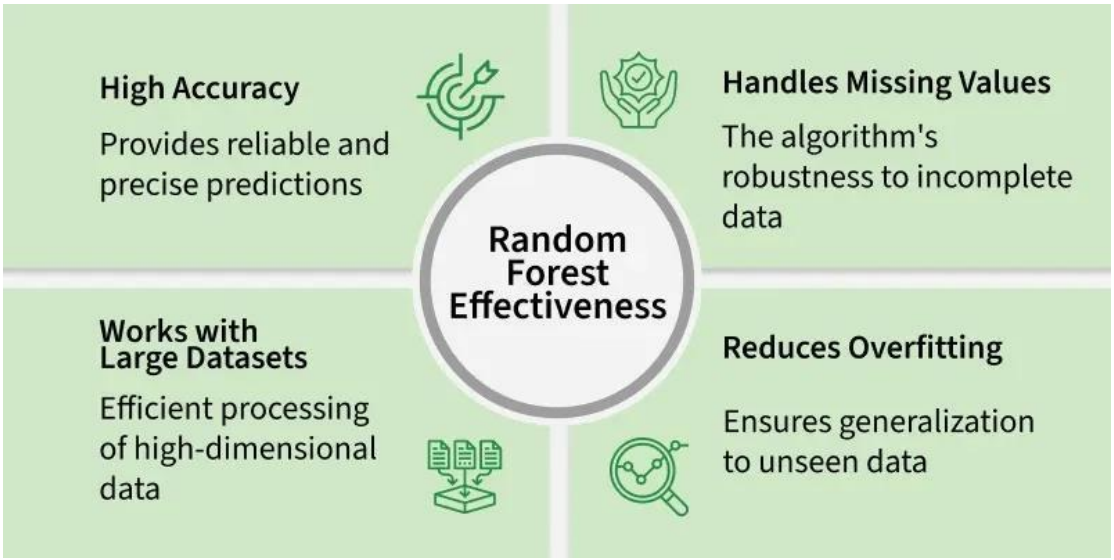
Ensemble Learning Techniques

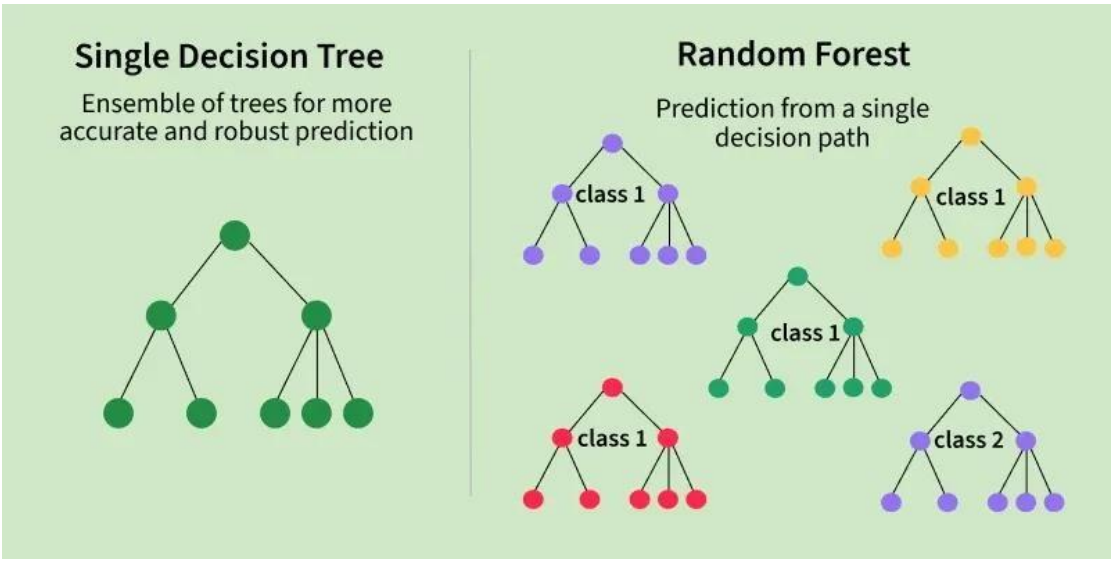
Technique	Category	Description
Random Forest	Bagging	Random forest constructs multiple decision trees on bootstrapped subsets of the data and aggregates their predictions for final output, reducing overfitting and variance.
Random Subspace Method	Bagging	Trains models on random subsets of input features to enhance diversity and improve generalization while reducing overfitting.
Gradient Boosting Machines (GBM)	Boosting	Gradient Boosting Machines sequentially builds decision trees, with each tree correcting errors of the previous ones, enhancing predictive accuracy iteratively.
Extreme Gradient Boosting (XGBoost)	Boosting	XGBoost do optimizations like tree pruning, regularization and parallel processing for robust and efficient predictive models.
AdaBoost (Adaptive Boosting)	Boosting	AdaBoost focuses on challenging examples by assigning weights to data points. Combines weak classifiers with weighted voting for final predictions.
CatBoost	Boosting	CatBoost specialize in handling categorical features natively without extensive preprocessing with high predictive accuracy and automatic overfitting handling.

Random Forest Algorithm in Machine Learning

Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it as ensemble learning technique. This helps in improving accuracy and reducing errors.







Working of Random Forest Algorithm

- **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Pick Random Features:** When building each tree it doesn't look at all the features (columns) at once. It picks a few at random to decide how to split the data. This helps the trees stay different from each other.
- **Each Tree Makes a Prediction:** Every tree gives its own answer or prediction based on what it learned from its part of the data.
- **Combine the Predictions:** For classification, the final answer is the category that most trees vote for (majority voting).
- **Why It Works Well:** Using random data and features for each tree helps avoid overfitting and makes the overall prediction more accurate and trustworthy.

Implementing Random Forest for Classification Tasks

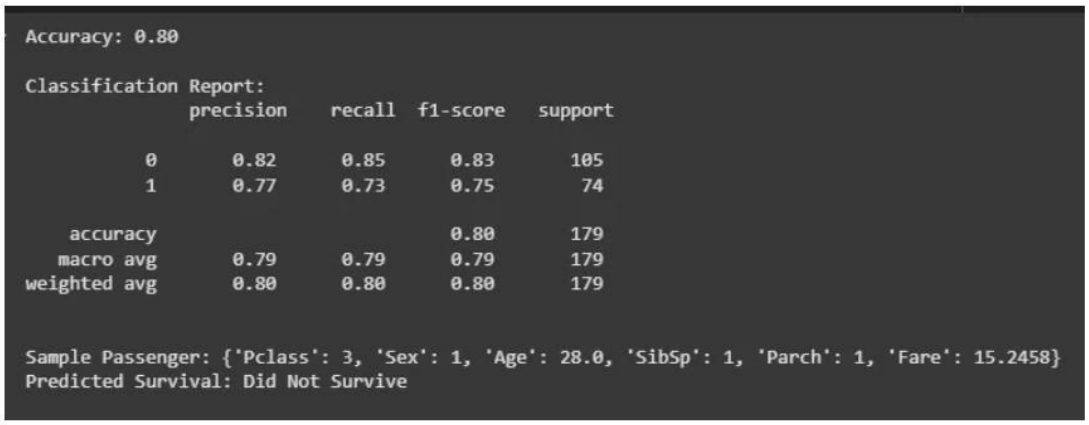
Here we will predict survival rate of a person in titanic.

- Import libraries like pandas and scikit learn.
- Load the Titanic dataset.
- Remove rows with missing target values ('Survived').

- Select features like class, sex, age, etc and convert 'Sex' to numbers.
- Fill missing age values with the median.
- Split the data into training and testing sets, then train a Random Forest model.
- Predict on test data, check accuracy and print a sample prediction result.

```
import pandas as pd from sklearn.model_selection import
train_test_split from sklearn.ensemble import
RandomForestClassifier from sklearn.metrics import
accuracy_score, classification_report import warnings
warnings.filterwarnings('ignore')

titanic_data = pd.read_csv('titanic.csv')

titanic_data = titanic_data.dropna(subset=['Survived'])

X = titanic_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']] y =
titanic_data['Survived']

X['Sex'] = X['Sex'].map({'female': 0, 'male': 1})
X['Age'] = X['Age'].fillna(X['Age'].median())

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)

y_pred = rf_classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy:.2f}") print("\nClassification
Report:\n", classification_rep)

sample = X_test.iloc[0:1] prediction =
rf_classifier.predict(sample)

sample_dict = sample.iloc[0].to_dict() print(f"\nSample Passenger: {sample_dict}")
print(f"Predicted Survival: {'Survived' if prediction[0] == 1 else 'Did Not Survive'}")
```

Output



Random Forest for Classification Tasks

Advantages

- Random Forest provides very accurate predictions even with large datasets.
- Random Forest can perform well after handling missing data through preprocessing techniques like imputation.
- It doesn't require normalization or standardization on dataset.
- When we combine multiple decision trees it reduces the risk of overfitting of the model.

Limitations

- It can be computationally expensive especially with a large number of trees.
- It's harder to interpret the model compared to simpler models like decision trees.

Gradient Boosting in ML

Gradient Boosting is a boosting algorithm and here each new model is trained to minimize the loss function such as mean squared error or cross-entropy of the previous model using gradient descent. In each iteration the algorithm computes the gradient of the loss function with respect to predictions and then trains a new weak model to predict this gradient. Predictions of the new model are then added to the ensemble (all models prediction) and the process is repeated until a stopping criterion is met.

Shrinkage and Model Complexity

A key feature of Gradient Boosting is shrinkage which scales the contribution of each new model using **learning rate** (denoted as $\eta\eta$).

- **Smaller learning rates:** mean the contribution of each tree is smaller which reduces the risk of overfitting but requires more trees to achieve the same performance.
- **Larger learning rates:** mean each tree has a more significant impact but this can lead to overfitting.

There's a trade off between the learning rate and the number of estimators (trees) a smaller learning rate usually means more trees are required to achieve optimal performance.

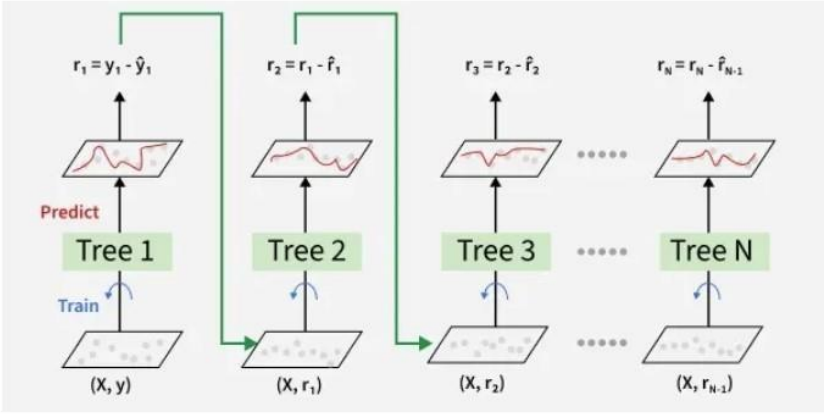
Working of Gradient Boosting

1. Sequential Learning Process

The ensemble consists of multiple trees each trained to correct the errors of the previous one. In the first iteration **Tree 1** is trained on the original data xx and the true labels yy. It makes predictions which are used to compute the errors.

2. Residuals Calculation

In the second iteration **Tree 2** is trained using the feature matrix xx and the errors from Tree 1 as labels. This means Tree 2 is trained to predict the errors of Tree 1. This process continues for all the trees in the ensemble. Each subsequent tree is trained to predict the errors of the previous tree.



Gradient Boosted Trees

3. Shrinkage

After each tree is trained its predictions are **shrunk** by multiplying them with the learning rate η which ranges from 0 to 1. This prevents overfitting by ensuring each tree has a smaller impact on the final model.

Once all trees are trained predictions are made by summing the contributions of all the trees. The final prediction is given by the formula:

$$\text{ypred}=\text{y1}+\eta\cdot\text{r1}+\eta\cdot\text{r2}+\cdots+\eta\cdot\text{rN} \quad y_{pred}=y_1+\eta\cdot r_1+\eta\cdot r_2+\cdots+\eta\cdot r_N$$

Where r1,r2,…,rN$r_1,r_2,\ldots,r_N$ are the errors predicted by each tree.

Example 1: Classification

We use Gradient Boosting Classifier to predict digits from Digits dataset.

- Import the necessary libraries
- Setting SEED for reproducibility
- Load the digit dataset and split it into train and test.
- Instantiate Gradient Boosting classifier and fit the model.
- Predict the test set and compute the accuracy score. from sklearn.ensemble import GradientBoostingClassifier from sklearn.model_selection import train_test_split from sklearn.metrics import accuracy_score

```
from sklearn.datasets import load_digits

SEED = 23

X, y = load_digits(return_X_y=True)

train_X, test_X, train_y, test_y = train_test_split(X, y,
                                    test_size = 0.25,
random_state = SEED)

gbc = GradientBoostingClassifier(n_estimators=300,
```

```
                    learning_rate=0.05,
random_state=100,
                    max_features=5 )

gbc.fit(train_X, train_y)

pred_y = gbc.predict(test_X)

acc = accuracy_score(test_y, pred_y) print("Gradient Boosting
Classifier accuracy is : {:.2f}".format(acc))
```

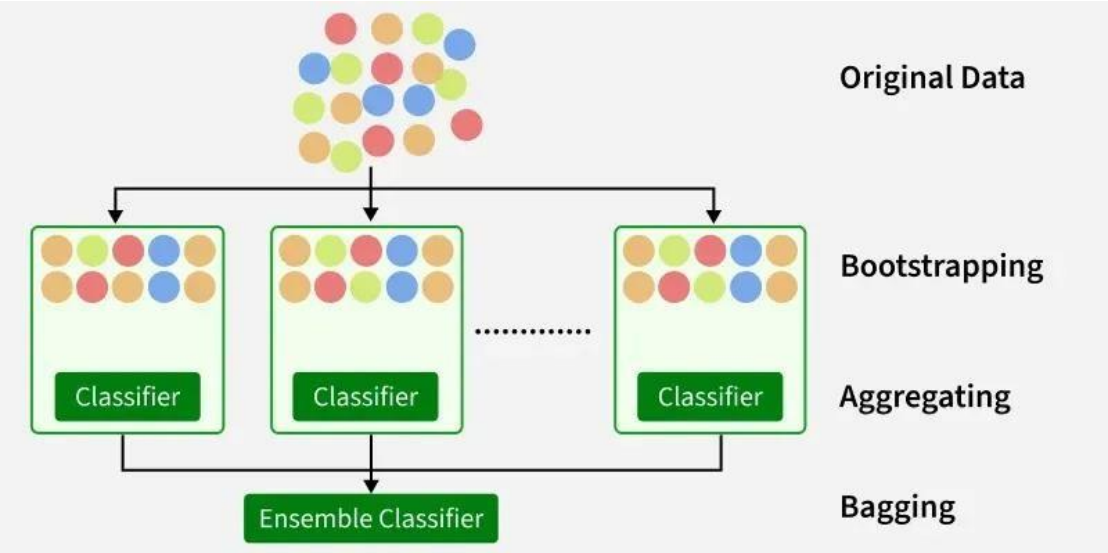
Output:

```
Gradient Boosting Classifier accuracy is : 0.98
```

Bagging Classifier

Bagging or Bootstrap Aggregating, works by training multiple base models independently and in parallel on different random subsets of the training data. These subsets are created using bootstrap sampling, where data points are randomly selected with replacement, allowing some samples to appear multiple times while others may be excluded.

- In classification tasks, the final prediction is decided by majority voting, the class chosen by most base models.
- For regression tasks, predictions are averaged across all base models, known as bagging regression.
- Bagging is versatile and can be applied with various base learners such as decision trees, support vector machines or neural networks.
- Ensemble learning broadly combines multiple models to create stronger predictive systems by leveraging their collective strengths.



Starting with an original dataset containing multiple data points (represented by colored circles). The original dataset is randomly sampled with replacement multiple times. This means that in each sample, a data point can be selected more than once or not at all. These samples create multiple subsets of the original data.

- For each of the bootstrapped subsets, a separate classifier (e.g., decision tree, logistic regression) is trained.
- The predictions from all the individual classifiers are combined to form a final prediction. This is often done through a majority vote (for classification) or averaging (for regression).

Bagging helps improve accuracy and reduce overfitting especially in models that have high variance.

Working of Bagging Classifier

- **Bootstrap Sampling**: From the original dataset, multiple training subsets are created by sampling with replacement. This generates diverse data views, reducing overfitting and improving model generalization.

  Let's break it down step by step:
  Original training dataset: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
  Resampled training set 1: [2, 3, 3, 5, 6, 1, 8, 10, 9, 1]
  Resampled training set 2: [1, 1, 5, 6, 3, 8, 9, 10, 2, 7] Resampled training set 3: [1, 5, 8, 9, 2, 10, 9, 7, 5, 4]

- **Base Model Training:** Each bootstrap sample trains an independent base learner (e.g., decision trees, SVMs, neural networks). These "weak learners" may not perform well alone but contribute to ensemble strength. Training happens in parallel, making bagging efficient.
- **Aggregation**: Once trained, each base model generates predictions on new data. For classification, predictions are combined via majority voting; for regression, predictions are averaged to produce the final outcome.
- **Out-of-Bag (OOB) Evaluation**: Samples excluded from a particular bootstrap subset (called out-of-bag samples) provide a natural validation set for that base model. OOB evaluation offers an unbiased performance estimate without additional cross-validation.

Applications

- **Fraud Detection**: Enhances detection accuracy by aggregating predictions from multiple fraud detection models trained on different data subsets.
- **Spam Filtering**: Improves spam email classification by combining multiple models trained on different samples of spam data.
- **Credit Scoring**: Boosts accuracy and robustness of credit scoring systems by leveraging an ensemble of diverse models.
- **Image Classification**: Used to increase classification accuracy and reduce overfitting by averaging results from multiple classifiers.
- Natur**al Language Processing (NLP)**: Combines predictions from multiple language models to improve text classification and sentiment analysis tasks.

Advantages

- Combines multiple models to reduce overfitting and improve accuracy.
- Reduces the impact of noise and outliers, making results more stable.
- Lowers variance by training on different samples, improving generalization.
- Works with various models like decision trees, SVMs, and neural networks.

Disadvantages

- Does not reduce bias, so errors remain if base models are biased.
- Can still overfit if base models are too complex.
- Offers limited improvement for already stable models.
- Requires proper tuning of parameters for best results.

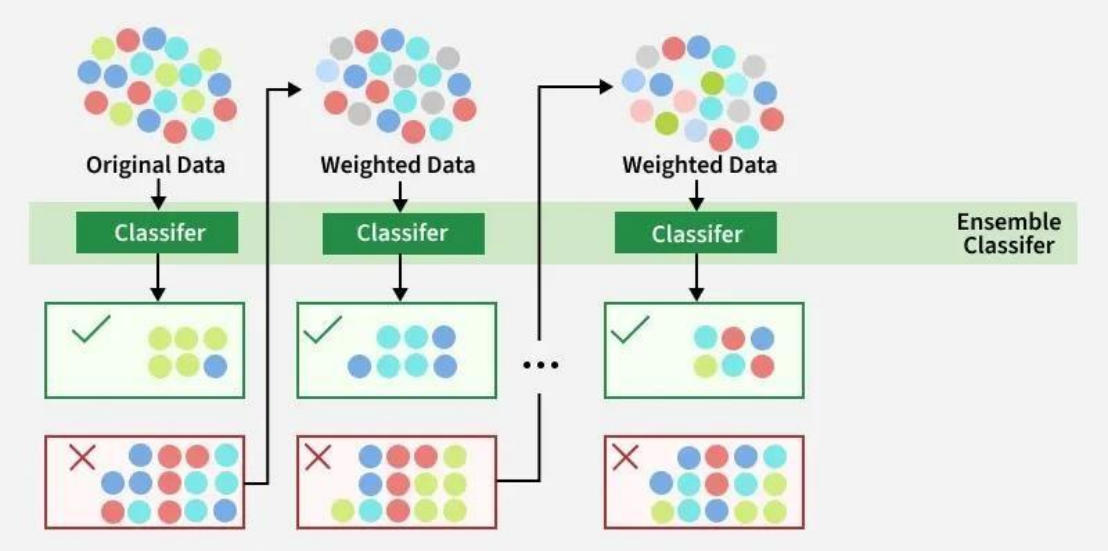
AdaBoost in Machine Learning

AdaBoost (Adaptive Boosting) is an ensemble learning technique that combines multiple weak classifiers to build a strong model. It works by sequentially focusing more on the misclassified data points from previous models.

- Trains models sequentially, each correcting previous errors
- Assigns higher weights to misclassified samples
- Final prediction is made using weighted voting

Adaboost Working

AdaBoost (Adaptive Boosting) assigns equal weights to all training samples initially and iteratively adjusts these weights by focusing more on misclassified data points for the next model. It effectively reduces bias and variance making it useful for classification tasks but it can be sensitive to noisy data and outliers.



The above diagram explains the AdaBoost algorithm in a very simple way. Let's try to understand it in a stepwise process:

Step 1: Initial Model (B1)

- The dataset consists of multiple data points (red, blue and green circles).
- Equal weight is assigned to each data point.
- The first weak classifier attempts to create a decision boundary.
- 8 data points are wrongly classified.

Step 2: Adjusting Weights (B2)

- The misclassified points from B1 are assigned higher weights (shown as darker points in the next step).
- A new classifier is trained with a refined decision boundary focusing more on the previously misclassified points.
- Some previously misclassified points are now correctly classified.
- 6 data points are wrongly classified.

Step 3: Further Adjustment (B3)

- The newly misclassified points from B2 receive higher weights to ensure better classification.

- The classifier adjusts again using an improved decision boundary and 4 data points remain misclassified.

Step 4: Final Strong Model (B4 - Ensemble Model)

- The final ensemble classifier combines B1, B2 and B3 to get strengths of all weak classifiers.
- By aggregating multiple models the ensemble model achieves higher accuracy than any individual weak model.

Now that we have learned how boosting works using Adaboost now we will learn more about different types of boosting algorithms.

Types Of Boosting Algorithms

There are several types of boosting algorithms some of the most famous and useful models are as :

1. **Gradient Boosting:** Gradient Boosting constructs models in a sequential manner where each weak learner minimizes the residual error of the previous one using gradient descent. Instead of adjusting sample weights like AdaBoost Gradient Boosting reduces error directly by optimizing a loss function.
2. **XGBoost:** XGBoost is an optimized version of Gradient Boosting that uses regularization to prevent overfitting. It is faster, efficient and supports handling both numerical and categorical variables.
3. **CatBoost:** CatBoost is particularly effective for datasets with categorical features. It employs symmetric decision trees and a unique encoding method that considers target values, making it superior in handling categorical data without preprocessing.

Advantages of Boosting

- **Improved Accuracy**: By combining multiple weak learners it enhances predictive accuracy for both classification and regression tasks.
- **Robustness to Overfitting**: Unlike traditional models it dynamically adjusts weights to prevent overfitting.
- **Handles Imbalanced Data Well**: It prioritizes misclassified points making it effective for imbalanced datasets.
- **Better Interpretability**: The sequential nature of helps break down decision-making making the model more interpretable.

By understanding Boosting and its applications we can use its capabilities to solve complex real-world problems effectively.